

# Despliegue de Aurora EEG en AWS

## 1. Objetivo

El objetivo del despliegue fue construir una plataforma SaaS para adquisición y visualización de señales EEG en tiempo real utilizando:

- Backend web en Laravel
  - WebSocket realtime con Node.js
  - Base de datos MySQL
  - Infraestructura cloud en AWS
  - Contenedores Docker desplegados en ECS Fargate
- 

## 2. Arquitectura General

La arquitectura final quedó compuesta por:

- Amazon VPC
  - 2 subnets públicas
  - 2 subnets privadas
  - Internet Gateway (IGW)
  - NAT Gateway
  - Application Load Balancer (ALB)
  - ECS Cluster (Fargate)
  - Servicio Laravel
  - Servicio WebSocket Node.js
  - Amazon RDS MySQL
  - Route 53 + dominio HTTPS
  - Certificado SSL/TLS con ACM
-

## 3. Creación de la infraestructura de red

### 3.1 VPC

Se creó una VPC privada para aislar toda la infraestructura:

10.0.0.0/16

---

### 3.2 Subnets públicas

Las subnets públicas contienen:

- Application Load Balancer
- NAT Gateway

Ejemplo:

10.0.1.0/24

10.0.2.0/24

---

### 3.3 Subnets privadas

Las subnets privadas contienen:

- Contenedores ECS
- Base de datos RDS

Ejemplo:

10.0.11.0/24

10.0.12.0/24

---

## 3.4 Internet Gateway

Se añadió un Internet Gateway a la VPC para permitir tráfico externo hacia:

- ALB
  - NAT Gateway
- 

## 3.5 NAT Gateway

El NAT Gateway permite que los contenedores privados tengan salida a Internet para:

- instalar dependencias
- acceder a repositorios
- descargar imágenes Docker

sin exponerlos públicamente.

---

# 4. Backend Laravel

## 4.1 Contenerización

El backend Laravel se dockerizó utilizando:

- PHP 8.3 FPM
- Nginx
- Composer

Dockerfile simplificado:

```
FROM php:8.3-fpm
```

---

## 4.2 Configuración HTTPS

Se solucionaron problemas de mixed content y formularios HTTPS mediante:

### **AppServiceProvider**

```
URL::forceScheme('https');
```

### **TrustProxies**

Configuración de proxies AWS ALB:

```
protected $proxies = '*';
```

---

## 4.3 Variables de entorno ECS

Las variables se definieron directamente en ECS:

```
APP_ENV=production
APP_URL=https://aurora-eeg.com
SESSION_DRIVER=database
CACHE_STORE=database
```

---

## 4.4 Sesiones

Inicialmente se utilizó:

```
SESSION_DRIVER=file
```

pero generaba problemas en ECS.

Finalmente se migró a:

```
SESSION_DRIVER=database
```

creando las tablas:

```
php artisan session:table
```

php artisan cache:table

php artisan migrate

---

## 5. WebSocket Realtime

### 5.1 Objetivo

Permitir transmisión EEG en tiempo real desde dispositivos ESP32 hacia el panel web.

---

### 5.2 Servidor Node.js

Se desarrolló un servidor WebSocket utilizando:

- Express
- ws

Servidor principal:

```
const WebSocket = require('ws');
```

---

### 5.3 Endpoint WebSocket

El endpoint final quedó:

```
wss://aurora-eeg.com/ws
```

---

### 5.4 ECS Realtime Service

Se desplegó un segundo servicio ECS:

- aurora-realtime

escuchando internamente en:

3000

---

## 5.5 Listener ALB

Se creó un listener HTTPS en el ALB:

443 → /ws

redirigiendo tráfico al target group realtime.

---

# 6. ESP32 EEG

## 6.1 Función

El ESP32:

- recibe datos EEG
  - se conecta al WiFi
  - transmite datos mediante WebSocket
- 

## 6.2 Configuración final

```
const char* wsHost = "aurora-eeg.com";
```

```
const int wsPort = 443;
```

Con conexión segura:

```
websocket.beginSSL(wsHost, wsPort, "/ws");
```

---

## 6.3 Flujo

1. ESP32 obtiene EEG
  2. Envía JSON al WebSocket
  3. Node.js recibe los datos
  4. Broadcast realtime
  5. Frontend recibe actualización en vivo
- 

## 7. Base de datos

### Amazon RDS MySQL

Se desplegó:

- MySQL en RDS
  - acceso privado
  - únicamente accesible desde ECS
- 

## 8. Seguridad

### Security Groups

#### ALB

Permite:

80  
443

---

#### ECS

Permite tráfico únicamente desde ALB:

3000

80

---

## **RDS**

Permite únicamente acceso desde ECS:

3306

---

# **9. Dockerización**

## **Backend Laravel**

Contenedor:

- PHP-FPM
  - Nginx
  - Composer
- 

## **Realtime Node.js**

Contenedor:

- Node.js 20
  - Express
  - ws
- 

# **10. Problemas encontrados**



## 10.1 Mixed Content HTTPS

Problema:

form action HTTP dentro de HTTPS

Solución:

- forceScheme HTTPS
  - TrustProxies
  - APP\_URL correcta
- 

## 10.2 Sesiones en ECS

Problema:

SESSION\_DRIVER=file

No persistía correctamente entre tareas.

Solución:

SESSION\_DRIVER=database

---

## 10.3 WebSocket 404

Problema:

Cannot GET /ws

Solución:

- configurar path `/ws`
  - listener ALB correcto
  - target group realtime
-

# 11. Resultado Final

La plataforma quedó desplegada con:

- Frontend Laravel funcionando en HTTPS
  - Login y registro operativos
  - Base de datos MySQL en RDS
  - WebSockets en tiempo real
  - ESP32 transmitiendo EEG
  - ECS Fargate escalable
  - Arquitectura cloud desacoplada y modular
- 

# 12. Tecnologías utilizadas

- Laravel 13
- PHP 8.3
- Node.js
- WebSocket (ws)
- Docker
- ECS Fargate
- Application Load Balancer
- Amazon RDS
- Route 53
- ACM SSL/TLS
- ESP32
- MySQL
- Nginx